

ICT600 CORRECTED STUDENT SCHEME

July 2025

Question-ordered model answers and marking guide

Scope	July 2025 final examination
Arrangement	Answers follow the original paper from Question 1 through Part B. Coding, diagrams, and written responses are integrated once at their correct positions.
Assessment	100 marks: Part A 70, Part B 30
Status	Corrected student reference; equivalent technically correct answers may receive credit.

How to use this scheme

Question-order rule

Attempt the paper first. Then check the matching answer in this document. Each answer appears once, in the same order as the question paper.

- Equivalent correct wording, diagrams, algorithms, and code may receive full credit when they meet the stated requirements.
- For coding, award method marks when a minor syntax error does not invalidate an otherwise correct solution.
- For design or illustration questions, the labelled diagram is the model answer; prose alone is not treated as a complete design.

Part A

Question 1(a)

8 marks

Define Internet, Web Standards, Web Server, and Content Management System.

Mark allocation

Awardable element	Marks
Internet	2
Web standards	2
Web server	2
CMS	2
Total	8

Complete model answer

- Internet: the global network of interconnected networks.
- Web standards: shared specifications supporting interoperable and accessible web content.
- Web server: software or a system that receives HTTP requests and returns web resources.
- CMS: software that lets authorised users create, manage, and publish web content.

Question 1(b)

10 marks

Identify FIVE differences between a web browser and a web application.

Mark allocation

Awardable element	Marks
Nature	2
Purpose	2
Architecture role	2
Data handling	2
Scope	2
Total	10

Complete model answer

- Compare nature, purpose, architecture role, data handling, and scope. A browser is client software that retrieves/renders resources; a web application is software delivered through that browser to perform user tasks.

Question 2(a)**6 marks****Define HTML, CSS, and Web Development.****Mark allocation**

Awardable element	Marks
HTML	2
CSS	2
Web development	2
Total	6

Complete model answer

- HTML structures web content.
- CSS controls presentation and layout.
- Web development is the process of building and maintaining web sites or applications using client-side and/or server-side technologies.

Question 2(b)**6 marks****State THREE principles of good web design.****Mark allocation**

Awardable element	Marks
First principle and explanation	2
Second	2
Third	2
Total	6

Complete model answer

- Any three well-explained principles, such as consistency, clear navigation, responsiveness, readability, accessibility, or organised content.

Question 3(a) - CRUD statements**8 marks****Write complete Create, Read, Update and Delete SQL statements.****Mark allocation**

Awardable element	Marks
INSERT	2
SELECT	2
UPDATE with WHERE	2
DELETE with WHERE	2
Total	8

Complete model answer

```
INSERT INTO Students (StudentID, Name, Age, Major)
VALUES (1001, 'Aina Rahman', 21, 'Computer Science');

SELECT * FROM Students;
```

```

UPDATE Students
SET Age = 22
WHERE StudentID = 1001;

DELETE FROM Students
WHERE StudentID = 1001;

```

Question 3(b) - enrollment JOIN

10 marks

Display unique student names and majors for semester 2025S1.

Mark allocation

Awardable element	Marks
SELECT DISTINCT	2
Required columns	2
Tables and INNER JOIN	2
StudentID join condition	2
Semester filter	2
Total	10

Complete model answer

```

SELECT DISTINCT s.Name, s.Major
FROM Students AS s
INNER JOIN Enrollments AS e
ON e.StudentID = s.StudentID
WHERE e.Semester = '2025S1';

```

Question 4(a)

4 marks

Explain how a web framework simplifies development compared with building from scratch.

Mark allocation

Awardable element	Marks
First benefit	1
Second	1
Third	1
Fourth	1
Total	4

Complete model answer

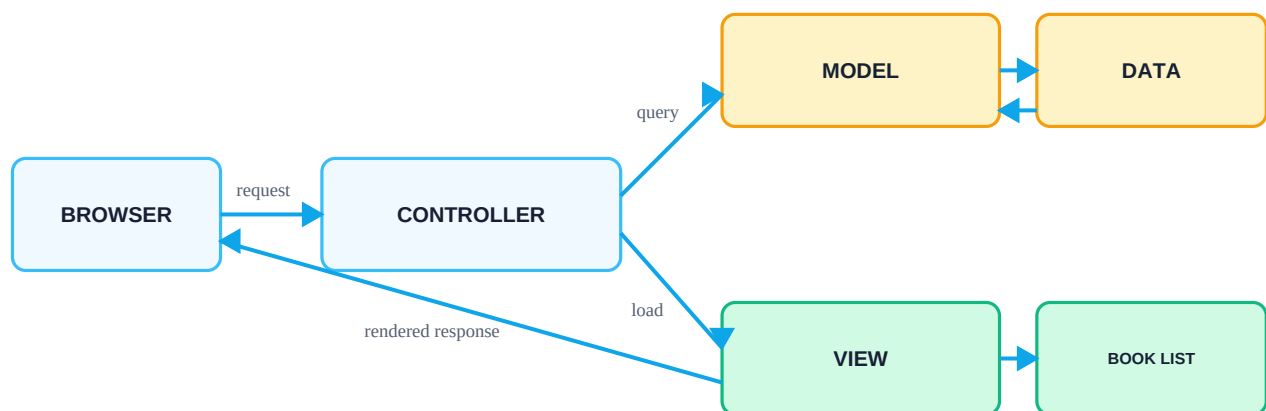
- Any four valid benefits: reusable components, routing, database/ORM facilities, security helpers, consistent structure, validation, templating, or faster development.

Question 4(b)**6 marks****Explain the MVC request flow for displaying a list of books.****Mark allocation**

Awardable element	Marks
Browser request	1
Controller action	2
Model/data retrieval	1
View rendering	1
Response to browser	1
Total	6

Complete model answer

- The controller receives the request, asks the model for book records, passes the returned data to the view, and the view renders the book list for the browser.

**Question 5(a)****6 marks****Identify THREE search-form security risks and a mitigation for each.****Mark allocation**

Awardable element	Marks
First risk and matching mitigation	2
Second	2
Third	2
Total	6

Complete model answer

- SQL injection - use prepared statements.
- Cross-site scripting - use context-aware output encoding.
- Data interception or leakage - use HTTPS/TLS and minimise exposed data.

Question 5(b)**6 marks****Describe THREE secure-coding techniques for sensitive user data.****Mark allocation**

Awardable element	Marks
Parameterized queries	2
Output encoding	2
Authentication/data-protection technique	2
Total	6

Complete model answer

- Parameterized queries prevent SQL injection.
- Context-aware output encoding prevents XSS.
- Password hashing, secure sessions, and TLS strengthen authentication and data protection.

Part B

Part B Question 1 - JavaScript validation

15 marks

Validate username and email and write the result to #message.

Mark allocation

Awardable element	Marks
Complete working validation behaviour	8
Syntax and readability comparison	2
Maintenance and scalability comparison	2
Beginner suitability	1
Justified recommendation	2
Total	15

Complete model answer

- Native JavaScript is used below. It avoids an extra dependency and is appropriate for this small form. jQuery offers concise selectors and historically strong cross-browser convenience, but adds a dependency and is less necessary for modern browser APIs. For larger projects, choose the approach that matches the existing architecture; for a new lightweight form, native JavaScript is the clearer recommendation.

```
const form = document.getElementById('registerForm');
const message = document.getElementById('message');

form.addEventListener('submit', event => {
  event.preventDefault();
  const username = document.getElementById('username').value.trim();
  const email = document.getElementById('email').value.trim();
  const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

  if (username === '' || email === '') {
    message.textContent = 'Username and email are required.';
  } else if (!emailPattern.test(email)) {
    message.textContent = 'Enter a valid email address.';
  } else {
    message.textContent = 'Registration Successful';
  }
});
```

Marker note

The remaining 7 marks are awarded for the written comparison: syntax/readability (2), maintenance/scalability (2), beginner suitability (1), and a justified recommendation (2).

Part B Question 2 - Student Result Analyzer

15 marks

Provide a runnable program that accepts a name and five marks, then calculates and displays total, average and grade.

Mark allocation

Awardable element	Marks
Name and five-mark input	2
Array/numeric validation	2
Total calculation	2
Average calculation	2
Correct grade boundaries	3
Complete output	2
Functions/control flow and valid structure	2
Total	15

Complete model answer

```
<!doctype html>
<html lang="en"><head><meta charset="utf-8"><title>Result Analyzer</title></head>
<body>
  <form id="resultForm">
    <label>Name <input id="studentName" required></label>
    <label>Five marks <input id="marks" placeholder="75,64,80,55,90" required></label>
    <button>Analyze</button>
  </form>
  <pre id="result"></pre>
  <script>
    document.getElementById('resultForm').addEventListener('submit', event => {
      event.preventDefault();
      const name = document.getElementById('studentName').value.trim();
      const marks = document.getElementById('marks').value.split(',').map(Number);
      if (!name || marks.length !== 5 || marks.some(m => !Number.isFinite(m) || m < 0 || m > 100)) {
        document.getElementById('result').textContent = 'Enter a name and five marks from 0 to 100.';
        return;
      }
      const total = marks.reduce((sum, mark) => sum + mark, 0);
      const average = total / marks.length;
      const grade = average >= 80 ? 'A' : average >= 70 ? 'B' :
        average >= 60 ? 'C' : average >= 50 ? 'D' : 'F';
      document.getElementById('result').textContent =
        `Student: ${name}\nMarks: ${marks.join(', ')}\nTotal: ${total}\n` +
        `Average: ${average.toFixed(2)}\nGrade: ${grade}`;
    });
  </script></body></html>
```